Parallel & Distributed Computing Lab

IMSciences

Lab Instructor: Saim Sanan

Week 5, Lab 1

Distributed computing intro and client–server sockets

---

1 Ideas

Until the parallel assignment, workers lived inside one process and could share an array (OpenMP / pthreads). Distributed computing uses separate processes (or machines) that cooperate by sending messages. There is no shared OpenMP array. This lab stays on one Ubuntu virtual machine.

- **Message passing** — processes send a copy of bytes (sockets now; MPI later).
- **Shared memory** — threads of one process share that process's memory.
- **Client–server** — one server waits; clients dial. This lab codes that pattern.
- **Peer-to-peer (P2P)** — equal peers; no single owner (for example BitTorrent).
- **Hybrid** — a server for control plus peers for heavy traffic (for example BitTorrent with a tracker).

`localhost` and port `12345` mean a door on this same machine. The server owns the port. The client visits it. Start the server first. If the client starts first, the connection is refused.

`encode` turns text into bytes for the network. `decode` turns bytes back into text. That is not encryption.

2 Commands

Open two terminals. Start the server in the first and leave it running. Start the client in the second.

```
nano server.py
python3 server.py
```

Second terminal:

```
nano client.py
python3 client.py
```

3 Server — `server.py`

```
import socket

server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
server.bind(("localhost", 12345))
server.listen(1)
print("Server listening on port 12345")

conn, addr = server.accept()
print("Connection from", addr)

data = conn.recv(1024).decode()
print("Received:", data)
```

```
conn.send("Hello from server!".encode())
conn.close()
server.close()
```

4 Client — `client.py`

```
import socket

client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
client.connect(("localhost", 12345))

client.send("Hello from client!".encode())
response = client.recv(1024).decode()
print("Server response:", response)

client.close()
```

Expected server lines include `Received: Hello from client!`. Expected client line: `Server response: Hello from server!`.

5 Check

1. State the difference between message passing and shared memory.
2. Name the three architectures and give one example of each. Which one does this lab code?
3. State what happens when the client starts before the server.